

IMPLEMENTATION PLAN

From Zero to GA

A six-phase, dependency-ordered build plan from project scaffold to signed release.

TABLE OF CONTENTS

What's Inside

— **Delivery Approach**

How six phases map to the Kanban-hybrid process in the SDLC document.

0 Foundation & Tooling

Repo, build pipeline, base schema, USDA ingestion.

1 Biometrics & Target Engine

Onboarding, BMR/TDEE, macro allocation, guardrails.

2 Local Database & Analytics Core

Quick lookup logging, weight tracking, the Dashboard.

3 Vision AI Calorie Logger

Ollama integration, verification UI, accuracy QA.

4 Smart Meal & Recipe Generator

Pantry recipes and macro-aware meal suggestions.

5 Polish, Hardening & Release

Cross-platform QA, accessibility, packaging, GA.

APPROACH

Delivery Approach

Development proceeds in six sequential phases, each ending in a working, demoable build rather than disconnected components. Phases are ordered by dependency risk: the offline, dependency-free features (biometrics, local database) ship first; the Ollama-dependent AI features — which carry the most technical uncertainty — are deliberately sequenced in the middle, once the application shell is stable enough to absorb their iteration.

6 DELIVERY PHASES	~17 ESTIMATED WEEKS TO GA	1-3 ENGINEERS ASSUMED	3 TARGET PLATFORMS AT GA
---	---	---	--

PHASE 0

Foundation & Tooling

Phase 0 — Foundation & Tooling

Weeks 1–2

- Scaffold Tauri v2 + React + TypeScript project; configure Tailwind CSS and shadcn/ui.
- Set up CI (build + unit test) for Windows, macOS, and Linux targets.
- Define base SQLite schema and wire tauri-plugin-sql with migration tooling.
- Build the USDA dataset ingestion script (raw CSV → pre-seeded usda_foods table + FTS5 index).
- Establish the Tauri CSP/capability config restricting network access to localhost:11434.

DELIVERABLE

Empty-shell app that launches on all 3 OSes with a searchable USDA table.

PHASE 1

Biometrics & Target Engine

Phase 1 — Biometrics & Target Engine

Weeks 3–4

- Implement BMI, BMR (Mifflin-St Jeor), and TDEE calculation as pure, unit-tested functions.
- Build the goal-oriented macro allocation engine for the four supported goals.
- Implement the Dietary Guardrail Matrix (diet patterns + allergy exclusions) and persistence.
- Build the onboarding wizard UI end-to-end (Welcome → Biometrics → Goal → Guardrails → Ready).

DELIVERABLE

A user can complete onboarding and see accurate personalized targets on a placeholder Dashboard.

PHASE 2

Local Database & Analytics Core

Phase 2 — Local Database & Analytics Core

Weeks 5–7

- Complete the full logging schema (food_log, food_log_items, weight_history).
- Build Quick Lookup logging against the pre-seeded USDA table (search → select → log).
- Build the real Dashboard: macro rings, remaining-budget bar, weight entry.
- Build Progress charts with Recharts: weight trend, macro compliance, logging streak.
- Implement CSV/JSON export.

DELIVERABLE

A fully usable manual tracking app — no AI dependency required yet — with working analytics and data export.

PHASE 3

Vision AI Calorie Logger

Phase 3 — Vision AI Calorie Logger

Weeks 8–11

- Build the Ollama REST client in Rust and the first-run system/model check flow.
- Design and iterate the structured-output prompt for llama3.2-vision against real meal photos.
- Implement the Zod (frontend) and serde (Rust) schema pair with the automatic re-prompt-on-failure path.
- Build the drag-and-drop / webcam capture UI and the editable Verification Table.
- Assemble the 40-photo golden-set library and the accuracy-scoring harness; tune prompts to the $\leq 15\%$ MAE target.

DELIVERABLE

End-to-end photo-to-logged-meal flow, with measured and documented accuracy against the golden set.

PHASE 4

Smart Meal & Recipe Generator

Phase 4 — Smart Meal & Recipe Generator

Weeks 12–14

- Integrate the local text LLM (llama3.2:3b / qwen2.5:7b) for recipe and meal-plan generation.
- Build the pantry ingredient input and the remaining-macro-budget scoring logic for candidates.
- Implement guardrail-constrained prompting so generated recipes always respect active diet/allergy rules.
- Build the recipe card UI and full-screen Cook Mode with step-by-step instructions.

DELIVERABLE

Users can generate a guardrail-safe recipe from pantry items and cook it end-to-end inside the app.

PHASE 5

Polish, Hardening & Release

Phase 5 — Polish, Cross-Platform Hardening & Release

Weeks 15–17

- Full pass against the UI/UX Brief: dark/light theme parity, motion polish, WCAG 2.1 AA audit.
- Cross-platform smoke testing on Windows, macOS (Apple Silicon & Intel), and Linux release candidates.
- Code signing/notarization, installer packaging (.msi, .dmg, .ApplImage/.deb), and updater manifest setup.
- Write user-facing onboarding docs and the in-app Privacy & Data page copy.
- Final Definition-of-Done review against every feature shipped in Phases 1–4.

DELIVERABLE

Signed, installable GA release published with changelog and auto-update enabled.

OVERVIEW

Timeline Summary

Phase	Focus	Weeks	Key Dependency
0	Foundation & Tooling	1-2	—
1	Biometrics & Target Engine	3-4	Phase 0 schema
2	Local Database & Analytics Core	5-7	Phase 1 targets
3	Vision AI Calorie Logger	8-11	Ollama runtime + Phase 2 logging schema
4	Smart Meal & Recipe Generator	12-14	Ollama runtime + Phase 1 guardrails
5	Polish, Hardening & Release	15-17	All prior phases feature-complete

PARALLELIZATION NOTE

With a 3-person team, Phase 4 (Recipe Generator) can run concurrently with the back half of Phase 3 once the Ollama client and Zod/serde contract pattern from Phase 3 are established — potentially compressing the timeline to ~14 weeks.

GATE

Exit Criteria per Phase

Phase	Must Be True to Proceed
0	CI green on all 3 OS targets; USDA table returns results in <5ms.
1	Calculated targets match reference Mifflin-St Jeor values within rounding for a 10-case test matrix.
2	A user can complete a full week of manual logging with accurate analytics and a working export.
3	Golden-set mean absolute calorie error ≤15%; zero unhandled schema-validation crashes across 200 test runs.
4	100% of generated recipes in a 50-sample guardrail-conflict test correctly respect active constraints.
5	Definition of Done satisfied for every P0/P1 requirement in the PRD; signed installers verified on all 3 OSes.